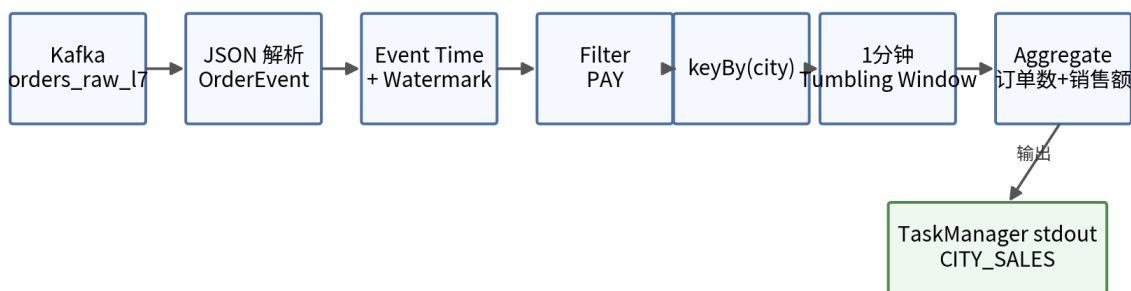


大数据综合项目实战

第 7 讲：Flink 接入 Kafka，实时统计城市销售额

Kafka Source → JSON → Event Time → Watermark → PAY → keyBy(city) → 1 分钟窗口 → 实时销售额

综合项目实时链：Kafka → Flink → Event Time → Window → 城市销售额



本讲目标：不是“让 Flink 能读 Kafka”就结束，而是必须证明：事件时间正确、Watermark 能推进、窗口能触发、乱序事件能被正确处理，并且最终金额与手工答案一致。

本讲继续使用上一讲的 Kafka 实时订单事件，但为了让实验结果完全可复现，我们单独使用教学 Topic：orders_raw_l7。生产系统通常会复用正式 Topic；教学实验为了避免旧数据干扰，采用隔离 Topic。

1. 本讲学完后，你必须真正会什么

- 知道 Kafka 与 Flink 在实时链中的分工：Kafka 保存事件，Flink 持续计算事件。
- 能启动并验证 Kafka、Flink、Java、Maven 环境，而不是直接盲目运行代码。
- 能解释 JSON 事件怎样变成 OrderEvent 对象，并从 event_time 提取真正的 Event Time。
- 能解释“最大乱序 5 秒”到底意味着什么，而不是把 Watermark 当成当前机器时间。
- 能解释为什么本项目必须先分配 Watermark，再过滤 PAY。
- 能解释 keyBy(city) 为什么会把同一城市的数据送到同一个逻辑分区。
- 能手算 1 分钟窗口的标准答案，并用标准答案验收 Flink 输出。
- 能通过 TaskManager 日志和 Flink Web UI 判断 Job 是否真的在运行。
- 能做一次迟到事件实验，理解“数据收到”与“窗口仍接受它”是两回事。

最低闭环：环境健康 → Topic 可用 → Job RUNNING → 事件进入 Kafka → Watermark 推进 → Window 输出 → 结果手算一致。只看到 RUNNING 远远不够。

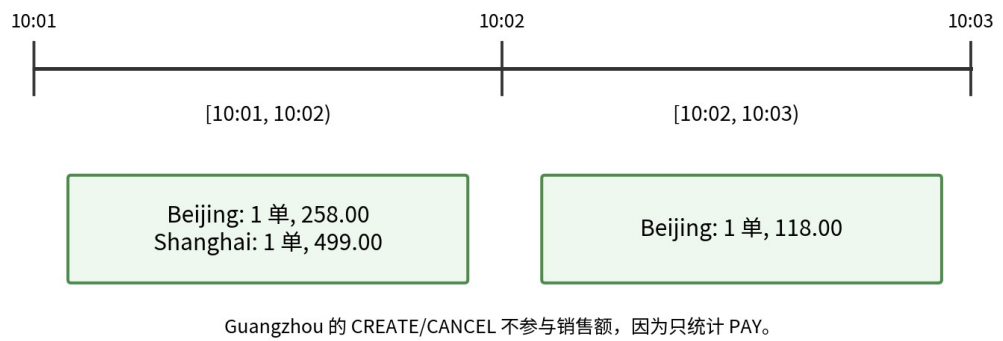
2. 先把业务问题说清楚：我们到底要实时算什么

需求：持续接收订单事件，只统计 event_type = PAY 的事件，按照 city 分组，每 1 分钟统计“支付订单数”和“支付销售额”。

字段	含义	本讲是否使用
event_id	事件唯一编号	用于观察
order_id	订单编号	Kafka Key；保证同订单事件尽量同分区
user_id	用户编号	本讲不分组
city	城市	keyBy 的 Key
amount	本次支付金额（元）	累计销售额
event_type	CREATE / PAY / CANCEL	只保留 PAY
event_time	业务事件实际发生时间	Event Time / Window 归属依据

金额为什么不用 double 累加：源码会把元转换成“分”的 long，例如 258.00 元 → 25800 分，再做整数累加，避免浮点数误差。

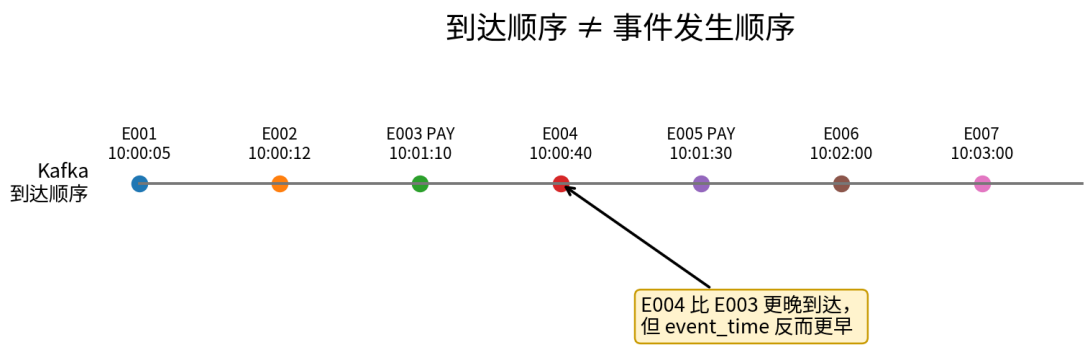
1 分钟 Event-Time 窗口的标准答案



窗口	城市	PAY 单数	销售额
[10:01:00, 10:02:00)	Beijing	1	258.00
[10:01:00, 10:02:00)	Shanghai	1	499.00
[10:02:00, 10:03:00)	Beijing	1	118.00

Guangzhou 的 O2004 只有 CREATE 和 CANCEL，没有 PAY，所以销售额结果中不应该出现 Guangzhou。

3. 为什么这一讲必须讨论 “乱序”



这就是为什么实时统计不能只看“到达系统的时间”。

在演示数据中，E003 的 event_time 是 10:01:10，但紧接着到达的 E004 的 event_time 却是 10:00:40。也就是说，Kafka 中“后到的记录”可以代表更早发生的业务事件。

核心区别：Arrival order 是数据到系统的顺序；Event Time 是业务事件真正发生的时间。实时业务统计通常更关心后者。

如果只按照 Processing Time 做 1 分钟窗口，那么网络延迟、重试、Kafka 堆积都会把本来属于 10:00 的事件错误地算到 10:01、10:02 甚至更晚的窗口。

4. Watermark：Flink 怎样判断 “这个窗口差不多可以结算了”

本讲采用一个非常适合入门的策略：允许事件最多乱序 5 秒。直觉模型可以写成：

Watermark ≈ 当前已经观察到的最大 Event Time - 5 秒

Watermark 推进示意：maxEventTime - 5 秒

到达事件	event_time	maxEventTime	Watermark	发生什么
E003 PAY	10:01:10	10:01:10	10:01:05	窗口暂不触发
E004 CREATE	10:00:40	10:01:10	10:01:05	乱序旧事件，WM不后退
E006 CREATE	10:02:00	10:02:00	10:01:55	[10:01,10:02) 还没关
E007 CANCEL	10:03:00	10:03:00	10:02:55	触发 [10:01,10:02)
E008 PAY	10:02:20	10:03:00	10:02:55	仍可进入 [10:02,10:03)
E009 CREATE	10:03:30	10:03:30	10:03:25	触发 [10:02,10:03)

Watermark 不是“当前机器时间”，而是对 Event Time 进度的估计。

非常关键：Watermark 只能向前推进，不会因为后面来了一个更旧的事件而倒退。

例如 E004 的 event_time=10:00:40 虽然比 E003 更旧，但此时 maxEventTime 已经是 10:01:10，因此 Watermark 仍然基于 10:01:10 计算，而不是退回 10:00:35。

本讲把 Watermark 放在 PAY 过滤之前。原因是 CREATE / CANCEL 虽然不参与销售额，但它们仍然能够提供“事件时间已经走到哪里”的证据。E009 本身不是 PAY，却专门用来把 Watermark 推过 10:03，从而关闭 [10:02,10:03) 窗口。

5. 开始上机前：固定目录与版本策略

建议在 VS Code Remote SSH 打开 Ubuntu 虚拟机后，把代码放在以下目录：

```
~/bigdata-project/lecture7-flink-kafka-city-sales/
```

代码包默认 Maven 配置使用 Flink 1.20.3、Kafka Connector 3.4.0-1.20、Java 11。之所以这样做，是为了使用稳定、经典的 DataStream API 与 KafkaSource，并降低 Flink 2.x / Java 17 对旧教学 VM 的迁移压力。

版本适配：先执行 `flink --version`。若你的 VM 中 Flink 不是 1.20.x，不要直接硬跑；应把 pom.xml 中 `flink.version` 与 `flink.kafka.connector.version` 改成与你实际环境匹配的组合。Hadoop 原来的 Java 8 不要永久替换，Flink 可在当前终端临时切换到 Java 11/17。

6. 第一步：启动前检查（每次实验都做）

打开 Ubuntu 虚拟机 → Windows 打开 VS Code → Remote SSH 连接 Hadoop 用户 → 打开远程目录 → Terminal → New Terminal。然后依次执行：

```
hostname
pwd
whoami
java -version
mvn -version
echo $KAFKA_HOME
echo $FLINK_HOME
$KAFKA_HOME/bin/kafka-topics.sh --version
$FLINK_HOME/bin/flink --version
```

看到什么才算正常：KAFKA_HOME / FLINK_HOME 不是空值；Kafka CLI 能显示版本；Flink CLI 能显示版本；Maven 可用；Flink 使用的 Java 版本满足你的 Flink 版本要求。

7. 第二步：启动并验证 Kafka

如果上一讲 Kafka 仍在运行，不要重复启动。先检查 9092：

```
ss -lntp | grep 9092 || true
jps
```

如果 9092 没有监听，再按照你当前 Kafka 安装方式启动 Broker。不同 Kafka 版本可能是 KRaft 或 ZooKeeper 模式，所以本讲不强行写死启动命令；重点是最终必须验证 localhost:9092 可用。

```
$KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
```

8. 第三步：为第 7 讲建立干净 Topic

为了避免上一讲 orders_raw 中已经存在的数据干扰窗口标准答案，本讲单独使用 orders_raw_l7。第一次建议手工执行：

```
$KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server localhost:9092 \
  --delete --topic orders_raw_l7 || true

sleep 2

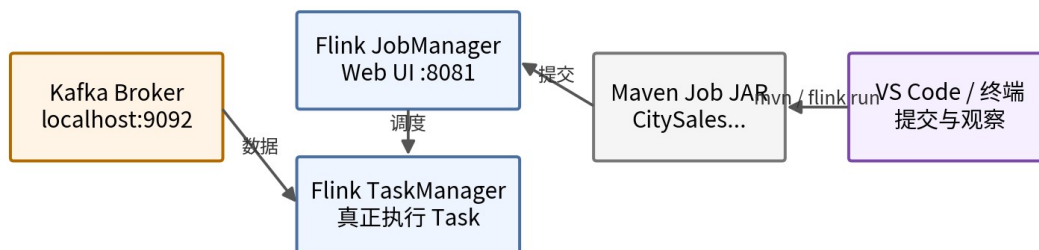
$KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server localhost:9092 \
  --create --if-not-exists --topic orders_raw_l7 \
  --partitions 3 --replication-factor 1

$KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server localhost:9092 \
  --describe --topic orders_raw_l7
```

验收：Topic 名称必须是 orders_raw_l7；PartitionCount=3；单机实验 ReplicationFactor=1。

9. 第四步：启动 Flink Standalone 集群并验证

单机训练环境中的运行组件



Kafka 负责“事件在哪里”，Flink 负责“事件怎样按时间持续计算”。

如果 Flink 还没启动：

```
$FLINK_HOME/bin/start-cluster.sh
```

```
jps
```

常见情况下你会看到 JobManager/StandaloneSessionClusterEntrypoint 与 TaskManager 相关进程。然后浏览器打开：

```
http://<虚拟机 IP>:8081
```

不要死记进程名：不同 Flink 版本的 JVM 进程名会变化。真正验收标准是：8081 Web UI 可访问，至少有 1 个 TaskManager，并且 Available Task Slots > 0。

10. 第五步：理解 Maven 项目结构

```
lecture7-flink-kafka-city-sales/
├── pom.xml
├── src/main/java/edu/project/realtime/
│   └── CitySalesRealtimeJob.java
├── data/
│   ├── order_events_window_demo.jsonl
│   └── late_event_after_watermark.jsonl
├── 07_emit_keyed_events.py
├── 07_reset_topic.sh
├── 07_send_demo_events.sh
├── 07_send_late_event.sh
└── 07_build_and_run.sh
```

第一次操作时，请把脚本当“参考答案”，不要上来直接一键执行。我们先把 Java 代码的每一段和前面学过的原理对上。

11. 第六步：KafkaSource - Flink 从哪里拿事件

```
KafkaSource<String> source = KafkaSource.<String>builder()
    .setBootstrapServers("localhost:9092")
    .setTopics("orders_raw_l7")
    .setGroupId("flink-city-sales-l7")
    .setStartingOffsets(OffsetsInitializer.earliest())
    .setValueOnlyDeserializer(new SimpleStringSchema())
    .build();
```

‘earliest()’的目的是让本讲每次从干净 Topic 的最早位置读取，便于复现实验。正式生产中通常会结合 Consumer Group 已提交 Offset、Checkpoint 与业务恢复策略来决定起点。

虽然 Source 这里只反序列化 value，但 Kafka Producer 仍然使用 order_id 作为 Kafka Key，因此同一个订单的 CREATE/PAY/CANCEL 会尽量被路由到同一个 Kafka Partition。

12. 第七步：JSON 解析与 Event Time 提取

```
JsonNode n = mapper.readTree(value);
...
e.eventType = n.get("event_type").asText();
e.eventTime = n.get("event_time").asText();

BigDecimal amountYuan = new BigDecimal(n.get("amount").asText());
e.amountCents = amountYuan.movePointRight(2).longValueExact();

LocalDateTime ldt = LocalDateTime.parse(e.eventTime, EVENT_TIME_FMT);
e.eventTimeMillis = ldt.atZone(BUSINESS_ZONE).toInstant().toEpochMilli();
```

为什么指定 Asia/Shanghai: 原始 event_time 字符串没有时区信息。为了让所有机器对“10:01:10”理解一致，本项目显式指定业务时区 Asia/Shanghai，而不是依赖服务器本地默认时区。

13. 第八步：真正的 Watermark 配置

```
WatermarkStrategy<OrderEvent> wm = WatermarkStrategy
    .<OrderEvent>forBoundedOutOfOrderness(Duration.ofSeconds(5))
    .withTimestampAssigner((e, ts) -> e.eventTimeMillis)
    .withIdleness(Duration.ofSeconds(20));

DataStream<OrderEvent> withWatermarks = parsed
    .assignTimestampsAndWatermarks(wm);
```

‘forBoundedOutOfOrderness(5s)’表示：系统愿意等一个“最多晚 5 秒”的乱序范围；‘withIdleness(20s)’是为以后多并行 Source 做准备，避免长时间没有数据的输入分区把下游 Watermark 永久卡住。

本讲教学简化：为了让 Watermark 推演清楚，代码把 env.setParallelism(1) 固定为 1。掌握后再改为 2/3，观察多个 Kafka Partition 与多个 Source Subtask 时的 Watermark 行为。

14. 第九步：过滤 PAY、keyBy(city)、1 分钟窗口

```
DataStream<OrderEvent> paid = withWatermarks
    .filter(e -> "PAY".equals(e.eventType));

DataStream<CitySalesResult> result = paid
    .keyBy(e -> e.city)
    .window(TumblingEventTimeWindows.of(Duration.ofMinutes(1)))
    .aggregate(new SalesAgg(), new AddWindowInfo());
```

这里把前面学过的概念真正串起来：Filter 决定哪些事件参与业务统计；keyBy(city) 决定同城市的数据必须“见面”；Tumbling Event-Time Window 决定每条 PAY 属于哪个 1 分钟时间桶；AggregateFunction 只保存“订单数 + 销售额”两个增量状态。

为什么不用把窗口内所有订单都存下来：因为我们只需要 COUNT 和 SUM。保存完整事件列表既浪费内存，也增加 State 和 Checkpoint 压力。能增量聚合，就不要保存全部历史。

15. 第十步：编译 Maven 项目

进入代码包目录后，先检查 pom.xml 中的 Flink 与 Connector 版本，再执行：

```
mvn -DskipTests clean package
```

成功后 target/ 中应该出现类似：

```
flink-kafka-city-sales-1.0-SNAPSHOT-all.jar
```

为什么是 all.jar：Kafka Connector 与 Jackson 不是 Flink 核心运行时自带的 API，本项目使用 Maven Shade Plugin 把这些运行依赖打进一个可提交的 Fat/Uber JAR；Flink core 本身仍以 provided 方式由集群提供。

16. 第十一步：提交 Flink Job

```
$FLINK_HOME/bin/flink run -d \  
-c edu.project.realtime.CitySalesRealtimeJob \  
target/flink-kafka-city-sales-1.0-SNAPSHOT-all.jar
```

记下返回的 JobID，然后执行：

```
$FLINK_HOME/bin/flink list -r
```

并进入 Web UI 的 Running Jobs 页面，应该能看到 `BigData Project L7 - Realtime City Sales`。

先启动 Job 再发数据：这样可以清楚看到“事件一条条进 Kafka → Flink 一条条消费 → Watermark 推进 → Window 触发”的全过程。

17. 第十二步：打开 TaskManager 输出日志

`result.print("CITY\_SALES")` 的输出发生在 TaskManager，而不一定显示在提交 Job 的终端。单机 Standalone 环境可以这样找到最新 stdout 文件：

```
TM_OUT=$(ls -lt "$FLINK_HOME"/log/*taskexecutor*.out 2>/dev/null | head -1)  
echo "$TM_OUT"  
tail -f "$TM_OUT"
```

如果你的版本没有匹配到 `.out`，可以先查看：

```
ls -lt $FLINK_HOME/log | head -20  
grep -R "CITY_SALES" $FLINK_HOME/log 2>/dev/null
```

18. 第十三步：发送 9 条演示事件

先不要执行脚本。打开 `data/order\_events\_window\_demo.jsonl`，确认 E003 后面紧跟着一个 event_time 更早的 E004，这就是我们故意准备的乱序。然后用辅助脚本逐条慢速发送：

```
./07_send_demo_events.sh
```

脚本内部把每行转换成：

```
order_id|完整 JSON
```

并让 kafka-console-producer 把 `|` 左边当 Kafka Key，右边当消息 Value。

19. 第十四步：观察 Watermark 为什么会在 E007 / E009 时触发窗口

Watermark 推进示意：maxEventTime - 5 秒

到达事件	event_time	maxEventTime	Watermark	发生什么
E003 PAY	10:01:10	10:01:10	10:01:05	窗口暂不触发
E004 CREATE	10:00:40	10:01:10	10:01:05	乱序旧事件，WM不后退
E006 CREATE	10:02:00	10:02:00	10:01:55	[10:01,10:02) 还没关
E007 CANCEL	10:03:00	10:03:00	10:02:55	触发 [10:01,10:02)
E008 PAY	10:02:20	10:03:00	10:02:55	仍可进入 [10:02,10:03)
E009 CREATE	10:03:30	10:03:30	10:03:25	触发 [10:02,10:03)

Watermark 不是“当前机器时间”，而是对 Event Time 进度的估计。

特别观察三个节点：

- E006 把 maxEventTime 推到 10:02:00，但 Watermark 约为 10:01:55，所以 [10:01,10:02) 还没有真正结束。
- E007 的 event_time=10:03:00，Watermark 约变成 10:02:55，于是 [10:01,10:02) 可以结算，输出 Beijing 258.00 和 Shanghai 499.00。
- E008 虽然在 E007 之后到达，但 event_time=10:02:20，仍属于还没关闭的 [10:02,10:03) 窗口，所以可以被接受。
- E009 是 CREATE，不参与销售额，但把 maxEventTime 推到 10:03:30，Watermark 超过 10:03，从而触发第二个窗口，输出 Beijing 118.00。

20. 第十五步：最终输出验收

TaskManager stdout 中输出的行顺序可能和下面不同，但业务集合必须等价：

```
CITY_SALES> window=[2026-09-29 10:01:00, 2026-09-29 10:02:00), city=Beijing, paid_orders=1, sales=258.00
CITY_SALES> window=[2026-09-29 10:01:00, 2026-09-29 10:02:00), city=Shanghai, paid_orders=1, sales=499.00
CITY_SALES> window=[2026-09-29 10:02:00, 2026-09-29 10:03:00), city=Beijing, paid_orders=1, sales=118.00
```

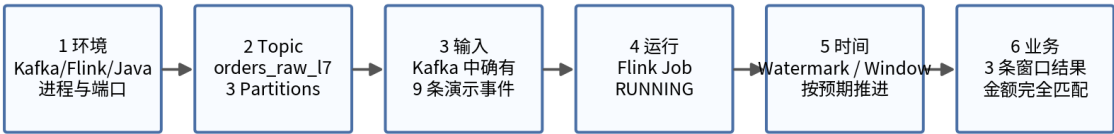
业务验收：不是“看起来差不多”，而是窗口边界、城市、订单数、销售额四个维度都必须和手算答案一致。

21. 第十六步：Web UI 到底看什么

打开 `http://<VM-IP>:8081` → 进入当前 Job。初学阶段重点看：

位置	看什么	异常意味着什么
Overview	Job 是否 RUNNING	FAILED / RESTARTING 先看日志
Job Graph	Kafka Source → parse → watermark → filter → window → sink	确认数据流拓扑是否如预期
Subtasks	每个算子的并行度	是否存在单个 Subtask 特别慢
Metrics	records in/out、busy、backpressure	判断数据是否真正流过
Exceptions	最近异常	JSON 解析、依赖、Kafka 连接等

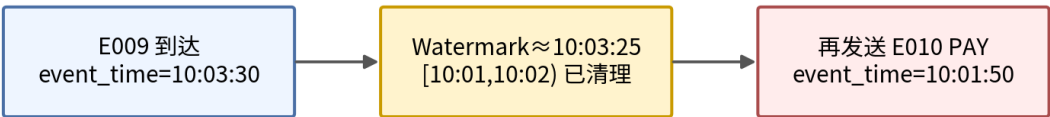
第7讲验收：必须拿到 6 层证据



只看到 Job=RUNNING，不等于实时业务逻辑正确。

22. 第十七步：迟到事件实验 - 为什么收到消息却不修正旧窗口

迟到实验：窗口已经关闭后再发送旧 PAY



默认 allowed lateness = 0：E010 不会修正已经输出并清理的旧窗口。

这不是“Flink 没收到数据”，而是时间语义告诉它：这个窗口已经结束。

等 E009 已经推动 Watermark 到 10:03 之后，再执行：

```
./07_send_late_event.sh
```

E010 是 PAY，金额 999.00，event_time=10:01:50。它从 Kafka 角度完全是“新收到的消息”，但它所属的 [10:01,10:02) 窗口早已被 Watermark 关闭并清理。默认 allowed_lateness=0 时，它不会让旧结果重新变成 Beijing 1257.00。

科研式观察：这个实验帮助你区分三个问题：Kafka 是否收到消息？Flink Source 是否消费消息？Window 是否仍接受这条 Event-Time 事件？这三个问题不是一回事。

23. 最常见的 8 类错误与固定排查顺序

现象	先查什么	典型原因
KafkaSource 无数据	Topic/9092/消息是否存在	Topic 名写错、Broker 未启动
Job 一提交就 FAILED	Exceptions + TaskManager log	Connector 依赖/Java 版本/JAR
JSON 解析异常	原始 Kafka Value	字段缺失、JSON 非法
窗口一直没输出	后续 Event Time 是否推进	没有足够新的事件推动 Watermark
只有部分城市	是否有 PAY	CREATE/CANCEL 被正确过滤
金额不对	amount 与事件类型	把 CREATE 也算进销售额
旧 PAY 不修正结果	当前 Watermark / allowed_lateness	事件已属于关闭窗口
Web UI RUNNING 但没业务结果	records in/out + 日志	RUNNING 只代表进程活着

固定顺序：环境 → Kafka → Source → JSON → Event Time/Watermark → Window → 聚合 → Sink/日志 → 业务答案。不要一上来先调内存。

24. 练习题

- 1. 为什么 E009 明明是 CREATE，却对第二个窗口能否输出非常重要？
- 2. 如果把 assignTimestampsAndWatermarks 放到 filter(PAY) 之后，本讲会出现什么不同？
- 3. 如果最大乱序时间从 5 秒改成 60 秒，窗口输出会更快还是更慢？为什么？
- 4. 为什么代码使用“分”的 long 累加金额，而不是直接累加 double？
- 5. 为什么同一个 order_id 作为 Kafka Key 有意义，但本讲 Flink 最终还是 keyBy(city)？

参考答案

- 1. E009 把 maxEventTime 推到 10:03:30，使 Watermark 超过 10:03，从而关闭 [10:02,10:03) 窗口；它不参与销售额，但参与时间进度判断。
- 2. 只有 PAY 事件才能推进 Watermark。如果后续长期没有 PAY，即使 CREATE/CANCEL 已经发生，窗口也可能迟迟不触发。
- 3. 通常更慢，因为系统要容忍更大的乱序范围，Watermark 会落后最大 Event Time 更多。

4. 二进制浮点对十进制金额可能出现精度误差；用“分”的整数累计更稳定。

5. Kafka Key 决定事件进入哪个 Kafka Partition，目标是尽量保持同一订单的事件局部有序；Flink keyBy(city) 是计算阶段的重新分区，目标是让同一城市的 PAY 进入同一聚合逻辑。

25. 本讲最终验收清单

- 能解释 Topic / Partition / Kafka Key 与 Flink keyBy 的区别。
- 能解释 Event Time 与 Arrival/Processing Time 的区别。
- 能手工推演 5 秒 bounded out-of-orderness 下的 Watermark。
- 能解释为什么 Watermark 在 PAY 过滤之前。
- 能独立编译 Fat JAR 并提交 Flink Standalone Job。
- 能找到 TaskManager stdout 并看到 CITY_SALES。
- 能核对三条标准窗口结果。
- 能完成 E010 迟到事件实验，并解释为什么旧窗口不修正。
- 遇到“不输出”时，先判断是 Kafka 没数据、Source 没读到，还是 Watermark 没推进。

本讲一句话：Kafka 负责让事件持续到达，Flink 负责按照业务事件时间持续计算；Watermark 决定“什么时候可以认为某个时间窗口基本结束”。

26. 下一讲预告

第 8 讲将在本讲实时链上继续增加“订单状态”。我们不再只做无状态的 PAY 统计，而是做：订单 CREATE 后 15 分钟仍没有 PAY → 自动告警。

Kafka → Flink → keyBy(order_id) → ValueState → Event-Time Timer → 超时告警

届时会把前面单独学过的 State、Timer、Checkpoint 真正接入这个综合项目。

参考依据

Apache Flink 官方文档：KafkaSource、WatermarkStrategy、Event-Time Window、DataStream Window Lifecycle。代码包采用经典 DataStream API，默认 Maven 版本写在 pom.xml 中，实际运行时应以你的 VM 已安装 Flink 版本为准。